=== UNIT 4: JAVASCRIPT LANGUAGE CORE, SCOPE & FUNCTIONS ===

>> 4.1 Evolution, Role and Runtime Environment of JavaScript

JavaScript (ECMAScript / ES) is the dynamic, high-level, interpreted scripting language that brings interactivity, programmatic logic, and asynchronous data transfer to web browsers.
* The Trinity of Front-End Web Development: HTML (Structure and Semantics), CSS (Presentation and Visual Style), JavaScript (Dynamic Behavior and User Interaction).
* JavaScript Engines and JIT Compilation: Modern browsers execute JS via high-performance engines (Google V8 in Chrome/Node.js, SpiderMonkey in Firefox, JavaScriptCore in Safari). These engines utilize Just-In-Time (JIT) compilation: source code is compiled into optimized machine bytecode at runtime rather than interpreted line-by-line.

>> 4.2 Variables, Hoisting and Data Types

* Variable Declarations: var, let, and const:
- var: Function-scoped, hoisted with default value undefined, permits re-declaration. Causes subtle bugs; legacy only.
- let (ES6): Block-scoped ({}), hoisted into Temporal Dead Zone (TDZ) where accessing before declaration throws ReferenceError, re-assignable.
- const (ES6): Block-scoped, TDZ-restricted, immutable reference binding (must be initialized upon declaration; primitives cannot change, but object properties can mutate).
* Data Types: Primitives (Number, BigInt, String, Boolean, Undefined, Null, Symbol) stored by value on the Call Stack; Objects (Object, Array, Function, Date, RegExp) stored by reference on the Memory Heap.
* Type Coercion: Loose equality (==) coerces types before checking (e.g. 5 == ’5’ is true); Strict equality (===) checks value AND type without coercion (5 === ’5’ is false). Always use ===.

>> 4.3 Functions, Arrow Syntax and Closures

* Function Declaration: Hoisted completely: function add(a, b) { return a + b; }.
* Function Expression: Not hoisted: const add = function(a, b) { return a + b; };.
* Arrow Functions (ES6): Lexically bind this keyword; concise syntax: const add = (a, b) => a + b;.

* Closures: A closure is the combination of a function bundled together with references to its surrounding lexical state (Lexical Scope). A closure gives an inner function access to an outer function's scope even after the outer function has finished executing and returned. Fundamental for data encapsulation and private variables.

>> 4.4 Array Higher-Order Methods (ES6+)

Modern JavaScript favors functional declarative array manipulation over manual for loops:
* .map(callback): Transforms each element, returning a new array of identical length.
* .filter(callback): Returns a new array containing only elements satisfying the test condition.
* .reduce(callback, initialValue): Combines array elements into a single accumulated result.
* .find(), .findIndex(), .some(), .every(), .includes(), .sort().

[PROGRAM 1: Data Type Validation and Strict Equality Engine]

Problem Context: Build a JavaScript module illustrating implicit type coercion versus strict equality, checking primitives vs objects, and handling null vs undefined.
JavaScript Source Code:

```
// Demonstrating JavaScript Type Coercion & Strict Equality
function demonstrateTypes() {
console.log('--- Type Coercion Analysis ---');
console.log('5' + 2);        // '52' (Number coerced to string)
console.log('5' - 2);        // 3    (String coerced to number)
console.log('5' * '2');      // 10   (Both coerced to numbers)
console.log([] == false);    // true (Complex coercion)
console.log([] === false);   // false (Different types: Object vs Boolean)

console.log('--- Null vs Undefined ---');
let unassignedVar;
let emptyVar = null;
console.log('unassignedVar: '${unassignedVar} ('${typeof unassignedVar})');
console.log('emptyVar:      '${emptyVar} ('${typeof emptyVar})');
console.log('null == undefined:  '${null == undefined}');   // true
console.log('null === undefined: '${null === undefined}');  // false
}
```

demonstrateTypes();
Key Learnings: typeof null returns 'object' (famous legacy JS bug); == treats null and undefined as loosely equal; === separates them strictly.

[PROGRAM 2: Factory Function with Private State via Closures]

Problem Context: Implement a secure bank account creator using JavaScript closures to maintain private encapsulated state without exposing variables to the global window scope.
JavaScript Source Code:

```
// Closure-based Private State Encapsulation
function createSecureAccount(accountHolder, initialDeposit) {
// Private variables inaccessible from external scope
let balance = initialDeposit;
let transactionHistory = [];

return {
getHolder: () => accountHolder
getBalance: () => balance
deposit: function(amount) {
if (amount <= 0) return 'Invalid deposit amount';
balance += amount;
transactionHistory.push({ type: 'DEPOSIT', amount, date: new Date() });
return 'Deposited INR '${amount}. New Balance: INR '${balance}';
}
withdraw: function(amount) {
if (amount > balance) return 'Insufficient funds';
balance -= amount;
transactionHistory.push({ type: 'WITHDRAWAL', amount, date: new Date() });
return 'Withdrew INR '${amount}. Remaining: INR '${balance}';
}
};
}

const acc = createSecureAccount('Aarav', 10000);
console.log(acc.deposit(5000));
console.log(acc.withdraw(2000));
console.log('Audited Balance: INR '${acc.getBalance()}');
console.log(acc.balance); // Output: undefined! Balance is completely private!
```

Execution Trace: Balance is modified only through closure methods; direct access

acc.balance yields undefined.

[PROGRAM 3: Functional Array Processing Pipeline (Map, Filter, Reduce)]

Problem Context: Given an array of student objects with course grades, filter students who passed, compute weighted grade point averages (GPA), and calculate class average using map, filter, and reduce.

JavaScript Source Code:

```
const students = [
{ id: 1, name: 'Aarav', marks: 88, credits: 4 }
{ id: 2, name: 'Diya',  marks: 42, credits: 4 }
{ id: 3, name: 'Rohan', marks: 94, credits: 3 }
{ id: 4, name: 'Kavita',marks: 76, credits: 3 }
];

// 1. Filter: Passing students (marks >= 50)
const passingStudents = students.filter(s => s.marks >= 50);

// 2. Map: Convert marks into 10-point GPA scale
const gpaCohort = passingStudents.map(s => ({
...s
gpa: Number((s.marks / 10).toFixed(1))
}));

// 3. Reduce: Compute overall cohort average GPA
const totalGPA = gpaCohort.reduce((acc, curr) => acc + curr.gpa, 0);
const averageGPA = (totalGPA / gpaCohort.length).toFixed(2);

console.log('Passing Students GPA:', gpaCohort);
console.log('Class Average GPA: '${averageGPA} / 10.0');
```

Execution Trace: Diya (42) filtered out. Passing cohort: Aarav (8.8), Rohan (9.4), Kavita (7.6). Average GPA = 8.60.

[PROGRAM 4: Object Destructuring, Rest & Spread Operators]

Problem Context: Implement user profile normalization illustrating ES6 object and array destructuring, default parameters, rest parameters, and spread cloning.

JavaScript Source Code:

```
// Modern ES6 Destructuring & Spread Syntax
const defaultPreferences = { theme: 'dark', notifications: true, language: 'en' };
const userRecord = { name: 'Aarav', email: 'aarav@lpu.in', theme: 'light' };

// Merge objects with spread operator (userRecord overrides defaults)
const userSettings = { ...defaultPreferences, ...userRecord };

// Object Destructuring with Aliasing and Defaults
const { name: userName, email, language = 'hi' } = userSettings;
console.log('User: '${userName} | Email: '${email} | Language: '${language}');

// Array Destructuring with Rest Parameters
const scores = [95, 88, 82, 74, 65];
const [topScore, secondScore, ...remainingScores] = scores;
console.log('Top Score: '${topScore}, Runner Up: '${secondScore}');
console.log('Other Scores: '${remainingScores}');
```

Output: Merges preferences smoothly; extracts topScore=95, secondScore=88, remainingScores=[82, 74, 65].

>> 4.5 University Review & Examination Questions

* Q1: Contrast 'var', 'let', and 'const' with respect to scope, variable hoisting, and the Temporal Dead Zone (TDZ).

Answer: 'var' is function-scoped; when declared inside an if-block or loop, it leaks into the enclosing function. 'var' declarations are hoisted to the top of their scope and initialized to 'undefined', allowing access before declaration without error. 'let' and 'const' (introduced in ES6) are block-scoped, existing strictly within their enclosing curly braces {}. While 'let' and 'const' are also hoisted by the JS engine during creation phase, they are NOT initialized. The time between entering the block scope and encountering the actual declaration statement is the Temporal Dead Zone (TDZ). Accessing a 'let' or 'const' variable in the TDZ throws a fatal ReferenceError. 'const' further mandates immediate initialization and forbids re-assignment.

* Q2: Explain the concept of Closures in JavaScript and demonstrate how closures can be used to emulate private object properties.

Answer: A closure is the persistent lexical scope bundle formed when an inner function retains access to variables declared in its outer enclosing function, even after that outer function has returned and exited the call stack. Because JavaScript historically lacked native 'private' class keywords, closures provide the canonical mechanism for data encapsulation. In a factory function, local variables defined inside the outer function cannot be accessed or modified directly by external code; they can only be read or manipulated via inner methods exposed in the returned public object, creating true

private variables.

>> 4.6 Key Takeaways and Revision Points

* Use const by default, let when re-assignment is required; never use legacy var.
* === enforces strict equality without implicit type coercion.
* Closures retain access to their outer lexical environment after execution completes.
* Arrow functions (() => {}) bind this lexically from their enclosing scope.